

Distributed Transactions

Exercise

Outline

In this exercise, we will build upon the apps created in the previous exercise to manage an order processing workflow, utilizing an event-driven architecture based on the Choreography pattern. The objective is to adapt the app to transition from the Choreography to the Orchestrator pattern, making the app more robust, maintainable, and promoting a more scalable order processing system.

At the end of this exercise, we will have a blueprint of the entire process and the flow of events that need to happen to support the order cycle, following the new strategy, as well as the necessary apps in ODC Studio to support it.

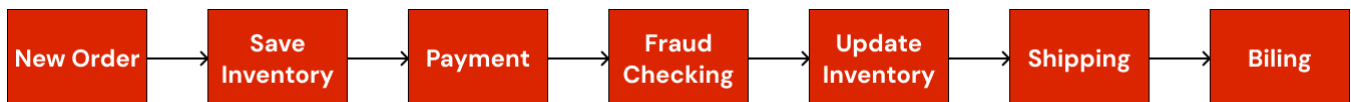
Use Case

Let's start by revisiting the use case and checking where we are after reshaping the original app to support the Event-Driven Architecture pattern.

Customer Portal

The EDA company is a very successful e-commerce company. They are always at the forefront of compliance as mandated by the local and international regulatory bodies, with very detailed verifications of every transaction.

Orders follow a standard cycle, as illustrated in the picture below. Each order progresses through multiple stages before being shipped and billed. Most steps occur sequentially, which makes it difficult to scale the process because server threads must wait for the entire procedure to complete.



In summary, when a new order is created, the process starts by checking the inventory and saving the products and quantity in the order. Then, there's the payment and verification to check for any fraud. If everything is ok, the inventory is updated, the order is shipped, and the customer is billed.

In the previous exercise, we enhanced the existing app by implementing the Event-Driven Architecture pattern, dividing the functionality into multiple apps, and utilizing Events to signal when a specific step of the workflow is completed.

Choreography Pattern

This implementation was based on the Choreography pattern to manage its order processing workflow. This involves multiple services (Fraud Detection, Billing, Inventory, etc) that independently react to Events to complete an order transaction. While this decentralized approach has benefits, it also presents challenges such as difficulty in tracking and managing the entire workflow, increased complexity in handling failures and maintaining data consistency across services.

The usage of the Choreography pattern in the current system leads to several issues:

- **Lack of Centralized Control:** Each service operates independently, making it hard to monitor and control the overall process.
- **Complex Error Handling:** Managing compensating transactions across multiple services becomes complex and error-prone.

- **Difficulty in Maintaining Consistency:** Ensuring data consistency across services is challenging, particularly in failure scenarios.
- **Scalability Issues:** As the number of services increases, the number of events and the complexity of managing interactions among services grow exponentially.

These challenges impact the reliability, maintainability, and scalability of the order-processing workflow.

Quickstart App

This exercise begins with the solution from the previous exercise. At this point, you should have the following apps (or similar):

- **EDA Storefront:** The app with the data model and Screens. This app has the logic to start the purchase and kickstart the workflow. This means checking the inventory, processing the payment, and triggering an event for the fraud verification.
- **EDA Inventory:** The app has the functionality to check if there is enough inventory to meet the order, and to block/pick the inventory after the fraud check is completed. When the inventory is selected, it triggers the shipping step.
- **EDA Fraud Detection:** This app has the logic to verify if the payment is fraudulent or not. After it is done, the logic triggers the step to block/pick the inventory.
- **EDA Shipping:** This app has the functionality to ship an order and to trigger the billing. Please note that the step to trigger the billing can also come from the EDA Inventory and be performed in parallel with the shipping.
- **EDA Billing:** This app has the logic to handle the order billing.
- **EDA Notification:** This app manages notifications for each step of the workflow.

Also note that many of these functionalities are based on empty wrappers, or in Sleep Actions, to simulate the actual verifications and steps. The most important factors lie in the organization of the apps and the workflow of events.

Hands-on

This exercise should be performed individually and has two parts: redesigning the workflow of events and improving the existing apps in ODC Studio.

Requirements and Suggestions

This central Orchestrator service approach aims to:

1. Provide centralized control and visibility of the workflow.
2. Simplify error handling and recovery processes.
3. Ensure better data consistency across services.
4. Enhance scalability by simplifying service interactions.

Redesign the diagram

Just like in the previous exercise, we should start by designing the workflow. Now, you already have the apps and the Events you need, but the workflow should be adapted to adopt the Orchestration pattern. So, revisit the workflow and change it to adapt to the new pattern.

Improve the apps

For this exercise, we do not need to divide the functionality into multiple apps, as we have already addressed that in the previous task. However, all the Events related to a step in the workflow should now be centralized within a single app.

In this centralized approach, it is essential to notify the orchestrator app when a specific step of the workflow is complete. Each step can either succeed or fail, and the logic must account for both outcomes. Additionally, each application needs to be aware of the compensation process and the subsequent steps in the workflow, even though this is outside their primary scope.

In line with the previous exercise, when making calls to external systems for payment processing, fraud detection, or sending notifications, there is no need to focus on the implementation details. You may create empty wrapper functions for these tasks. The same approach applies to compensation mechanisms as well.